

# Inżynieria obliczeniowa w analizie danych cyfrowych

## Projekt 3 - Podstawy Gymnasium

Kacper Gawroński, Paweł Gleindek

April 2026

### Spis treści

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>Gra z dyskretną przestrzenią akcji - Frozen Lake</b> | <b>2</b>  |
| 1.1      | Opis problemu . . . . .                                 | 2         |
| 1.2      | Metodyka oraz eksperymenty . . . . .                    | 2         |
| 1.2.1    | Eksperyment 1 . . . . .                                 | 4         |
| 1.2.2    | Eksperyment 2 . . . . .                                 | 5         |
| 1.2.3    | Eksperyment 3 . . . . .                                 | 6         |
| 1.2.4    | Eksperyment 4 . . . . .                                 | 7         |
| 1.2.5    | Eksperyment 5 . . . . .                                 | 8         |
| 1.2.6    | Eksperyment 6 . . . . .                                 | 9         |
| <b>2</b> | <b>Gra z ciągłą przestrzenią obserwacji - CarRacing</b> | <b>10</b> |
| 2.1      | Opis problemu . . . . .                                 | 10        |
| 2.2      | Metodyka oraz eksperymenty . . . . .                    | 10        |
| <b>3</b> | <b>Algorytmy uczenia ze wzmocnieniem</b>                | <b>12</b> |
| 3.1      | Wprowadzenie . . . . .                                  | 12        |
| 3.2      | Algorytm Q-Learning . . . . .                           | 12        |
| 3.3      | Algorytm Value Iteration . . . . .                      | 12        |
| 3.3.1    | Wstęp teoretyczny . . . . .                             | 12        |
| 3.3.2    | Analiza działania . . . . .                             | 14        |
| 3.4      | Algorytm Monte Carlo Control . . . . .                  | 17        |
| 3.4.1    | Wstęp teoretyczny . . . . .                             | 17        |
| 3.4.2    | Analiza działania . . . . .                             | 17        |
| 3.5      | Strojenie hiperparametrów . . . . .                     | 21        |
| 3.5.1    | Q-Learning . . . . .                                    | 21        |
| 3.5.2    | Value Iteration . . . . .                               | 22        |
| 3.5.3    | Monte Carlo Control . . . . .                           | 23        |
| 3.6      | Podsumowanie . . . . .                                  | 24        |

# 1 Gra z dyskretną przestrzenią akcji - Frozen Lake

## 1.1 Opis problemu

Jako grę, której przejścia będzie uczył się agent wybrano *Frozen Lake*. Środowisko **Frozen Lake** jest jednym z klasycznych problemów w bibliotece Gymnasium, symulującym poruszanie się agenta po zamrożonym jeziorze przedstawionym w formie siatki pól. Głównym celem gry jest bezpieczne przeprowadzenie agenta z pola startowego (*Start*) do pola końcowego (*Goal*), na którym znajduje się prezent. Jezioro składa się z trzech rodzajów pól: *powierzchni lodu*, *dziur*, do których wpadnięcie kończy się natychmiastową przegraną, oraz *prezentu*.

Środowisko to charakteryzuje się dyskretną przestrzenią obserwacji (numer pola od 0 do 15 dla siatki 4x4) oraz dyskretną przestrzenią akcji, gdzie agent może wykonać jeden z czterech ruchów: w lewo, w dół, w prawo lub w górę. Kluczową cechą **Frozen Lake** jest możliwość włączenia trybu śliskiego lodu (*is\_slippery=True*). W tym wariantcie środowisko przestaje być deterministyczne, co oznacza, że wykonanie danej akcji nie gwarantuje przemieszczenia się w wybranym kierunku — agent może poślizgnąć się i wylądować na sąsiednim polu, co znacząco utrudnia proces nauki i wymaga od agenta uwzględnienia niepewności ruchu. System nagród w tej grze jest następujący: 1 pkt. za przejście do prezentu, 0 za przegraną.

## 1.2 Metodyka oraz eksperymenty

W ramach tego zadania przeprowadzoną znaczącą liczbę eksperymentów, z różnymi parametrami agenta jak również z samym środowiskiem (mowa tu o wspomnianej wcześniej opcji *is\_slippery*).

Do procesu nauki wykorzystano algorytm **Q-learning**. Idea algorytmu jest prosta - używana jest tutaj macierz Q, której wierszami są stany w środowisku, a kolumnami możliwe akcje do podjęcia, wartością w danej komórce jest ocena jak dobry jest ten ruch w danej sytuacji. Macierz ta może być na początku zainicjalizowana odpowiednią ilością stanów i akcji, lub w miarę nauki nowe wpisy są dodawane - wtedy nie jest to już macierz, lecz słownik, ze stanem jako klucz - wykorzystywane między innymi w *black jacku* w tej bibliotece. Po każdym podjętym ruchu następuje aktualizacja macierzy Q zgodnie z równaniem Bellman'a, które jest sercem tego algorytmu, wzór jest następujący:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[ R + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (1)$$

Gdzie:

- $Q(s, a)$  – bieżąca wartość dla pary stan-akcja w macierzy Q,
- $\alpha$  (learning rate) – współczynnik uczenia określający stopień przyswajania nowych informacji,
- $R$  – nagroda otrzymana po wykonaniu akcji  $a$  w stanie  $s$ ,
- $\gamma$  (discount factor) – współczynnik dyskontowy,
- $\max_{a'} Q(s', a')$  – maksymalna przewidywana wartość nagrody dla następnego stanu  $s'$ .

Ostatnimi parametrami, które są równie ważne, lecz nie wchodzą w to równanie są:

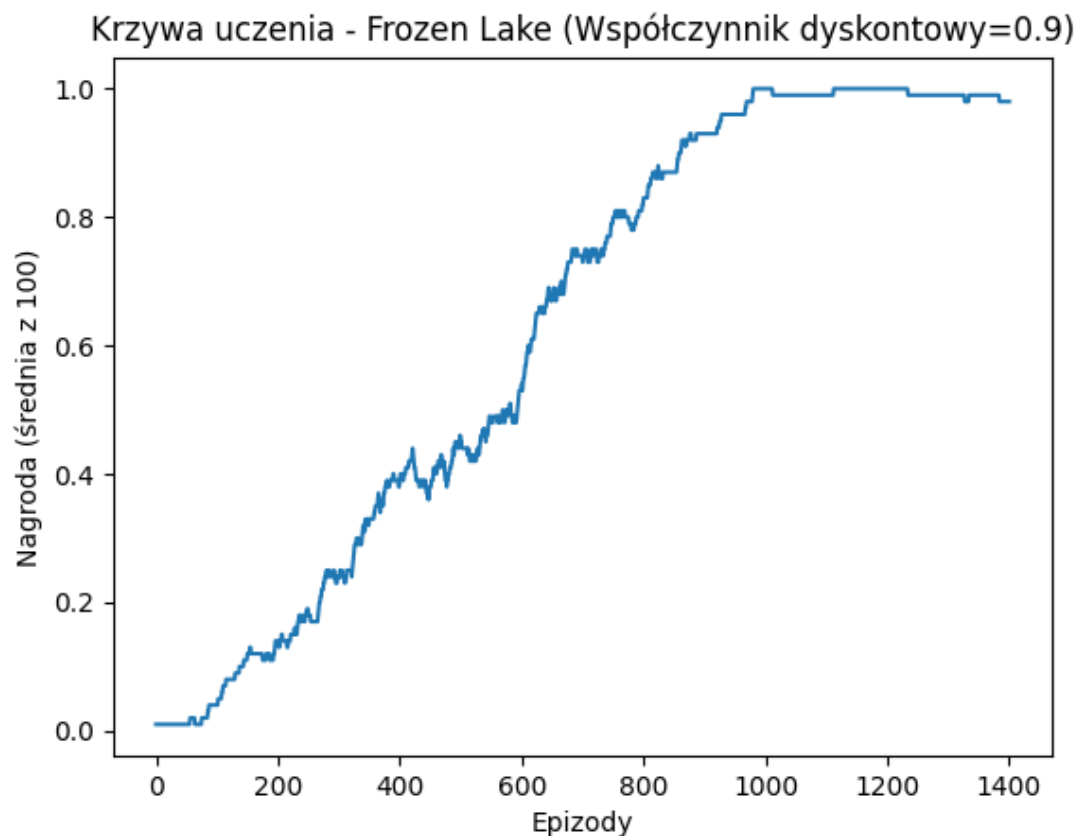
- $\epsilon$  - współczynnik odpowiedzialny za eksplorację i eksploatację, agent z prawdopodobieństwem  $\epsilon$  wybiera losową akcję a z prawdopodobieństwem  $1 - \epsilon$  wybiera akcję najlepszą w danej sytuacji, ma to na celu balansowaniem pomiędzy poznaniem najlepszych ruchów, a faktycznym zbliżeniu się do jak największej nagrody
- $\epsilon_{decay}$  - współczynnik odpowiedzialny za zmniejszanie się wartości  $\epsilon$  wraz z rosnącą ilością epizodów - czyli ilości pełnych przejść przez dane środowisko, ma to na celu zmaksymalizowanie osiągnięć agenta, poprzez wczesne skupienie się na eksploracji, a później na eksploatacji. co gwarantuje jak najlepsze osiągnięcia agenta podczas procesu uczenia

Agent podczas treningu przeszedł przez 1500 epizodów, gdyż w późniejszych epizodach zdolność agenta bardzo się wahała dla wariantu z poślizgiem. Nagroda dla agenta została wygładzona za pomocą okna o długości 100, w celu ustabilizowania wykresów.

### 1.2.1 Eksperyment 1

Parametry:

- $\alpha = 0.5$ ,
- $\text{discount\_level} = 0.9$ ,
- $\epsilon = 1$ ,
- $\text{eps\_decay} = 0.001$ ,
- $\text{is\_slippery} = \text{False}$ ,
- $\text{episodes} = 1500$



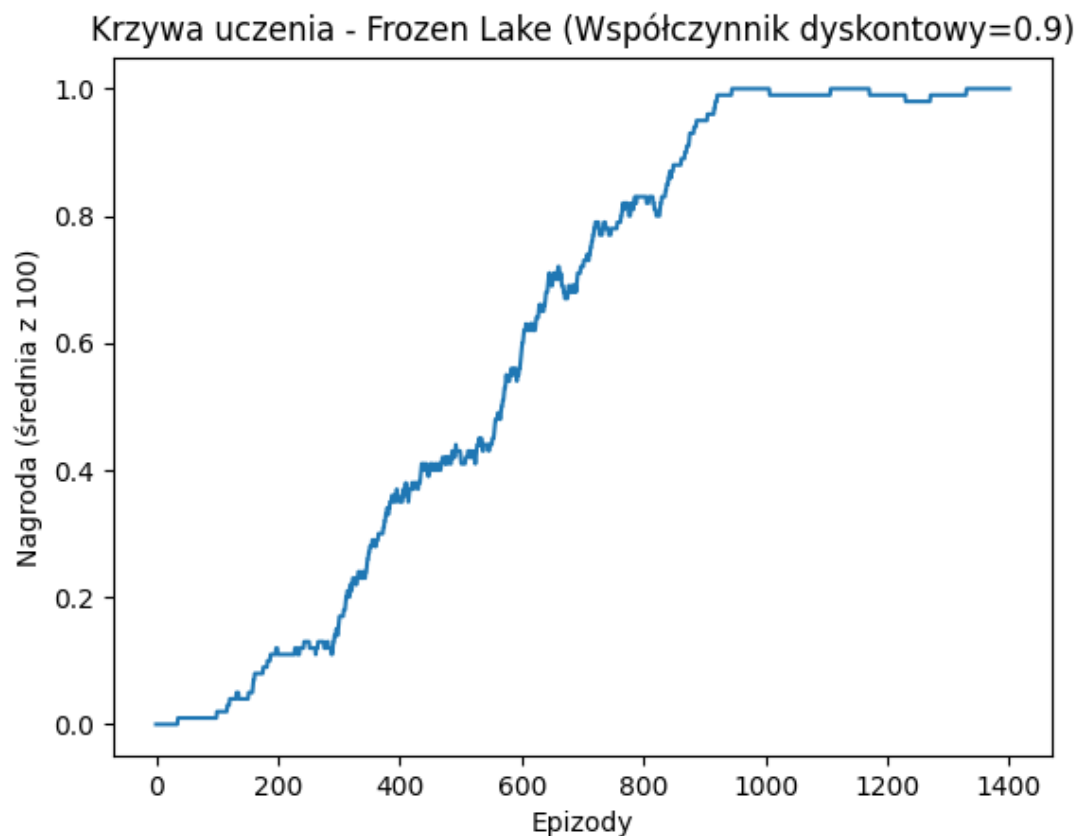
Rysunek 1: Krzywa uczenia dla eksperymentu 1

Jak widać w podstawowej wersji agent, już po około 800 epizodach perfekcyjnie opanował grę Frozen Lake w prostym wariancie - średnia nagroda ze 100 gier wynosiła wtedy mniej więcej 1 co oznacza, że praktycznie zawsze udawało mu się znaleźć prezent. Wykorzystano tutaj podstawowy, najbardziej powszechny i sprawny zestaw parametrów w algorytmie *Q-learning*, poza  $\alpha$ , które wynosiło tutaj, aż 0.5 co powinno powodować drastycznie szybkie uczenie, które mogłoby być niestabilne. Ten wariant jest jednak na tyle prosty, że nie powodowało to żadnych problemów.

### 1.2.2 Eksperyment 2

Parametry:

- $\alpha = 0.1$ ,
- $\text{discount\_level} = 0.9$ ,
- $\text{epsilon} = 1$ ,
- $\text{eps\_decay} = 0.001$ ,
- $\text{is\_slippery} = \text{False}$ ,
- $\text{episodes} = 1500$



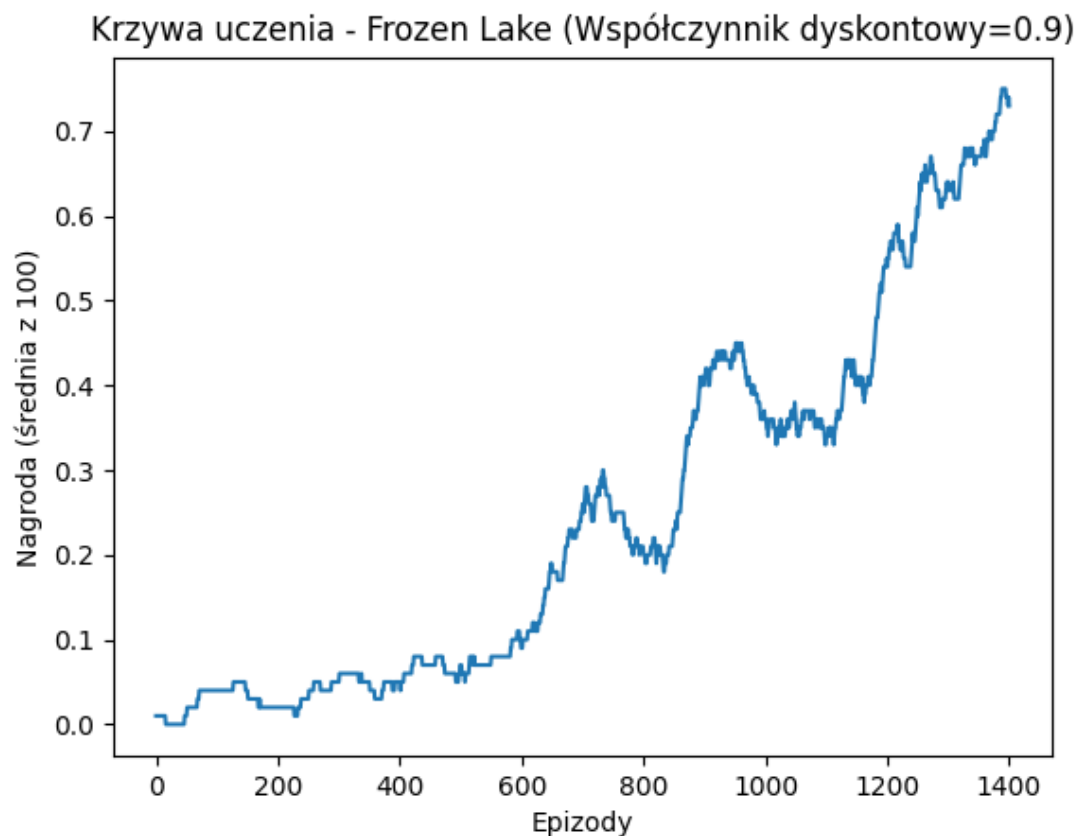
Rysunek 2: Krzywa uczenia dla eksperymentu 2

Jak widać w podstawowej wersji agent, już po około 800 epizodach perfekcyjnie opanował grę Frozen Lake w prostym wariantcie - średnia nagroda ze 100 gier wynosiła wtedy mniej więcej 1 co oznacza, że praktycznie zawsze udawało mu się znaleźć prezent. Wykorzystano tutaj podstawowy, najbardziej powszechny i sprawny zestaw parametrów w algorytmie *Q-learning*. Widać tutaj, że wcześniej zwiększony parametr  $\alpha$ , nie miał, aż tak dużego wpływu na szybkość nauki.

### 1.2.3 Eksperyment 3

Parametry:

- $\alpha = 0.1$ ,
- $\text{discount\_level} = 0.9$ ,
- $\epsilon = 1$ ,
- $\text{eps\_decay} = 0.001$ ,
- $\text{is\_slippery} = \text{True}$ ,
- $\text{episodes} = 1500$



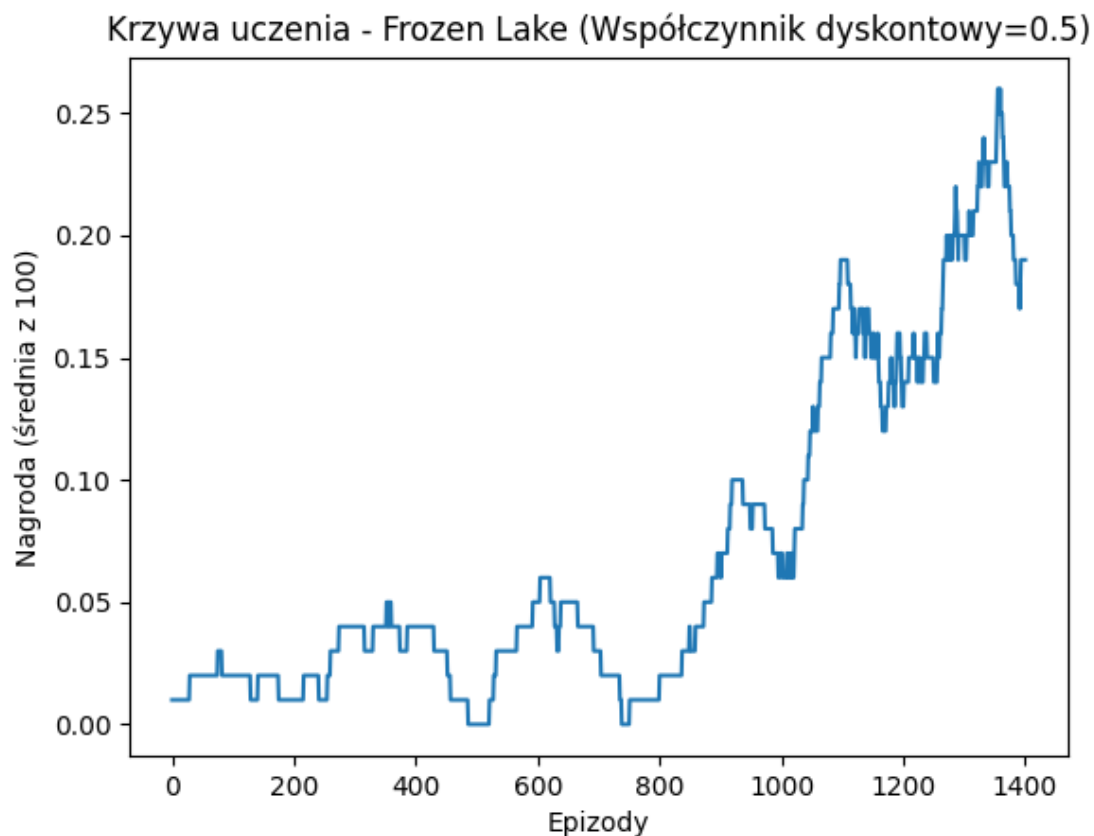
Rysunek 3: Krzywa uczenia dla eksperymentu 3

Jak widać w rozszerzonej wersji ( $\text{is\_slippery}=\text{True}$ ) agent, napotkał pewne trudności w nauce. Około epizodu 800 i 1000, doszło do dużego spadku w nagrodzie, natomiast przy końcu uczenia agent uzyskał zadowalający wynik 0.7. Zastosowano tutaj typowy zestaw parametrów, wspomniany wcześniej. Sama natura tego problemu uniemożliwia uzyskanie tutaj perfekcyjnego rezultatu - gra jest niedeterministyczna, co może doprowadzić do scenariuszy, w których nie da się zawsze wygrać, niezależnie od wykonanego ruchu.

### 1.2.4 Eksperyment 4

Parametry:

- $\alpha = 0.1$ ,
- $\text{discount\_level} = 0.5$ ,
- $\epsilon = 1$ ,
- $\text{eps\_decay} = 0.001$ ,
- $\text{is\_slippery} = \text{True}$ ,
- $\text{episodes} = 1500$



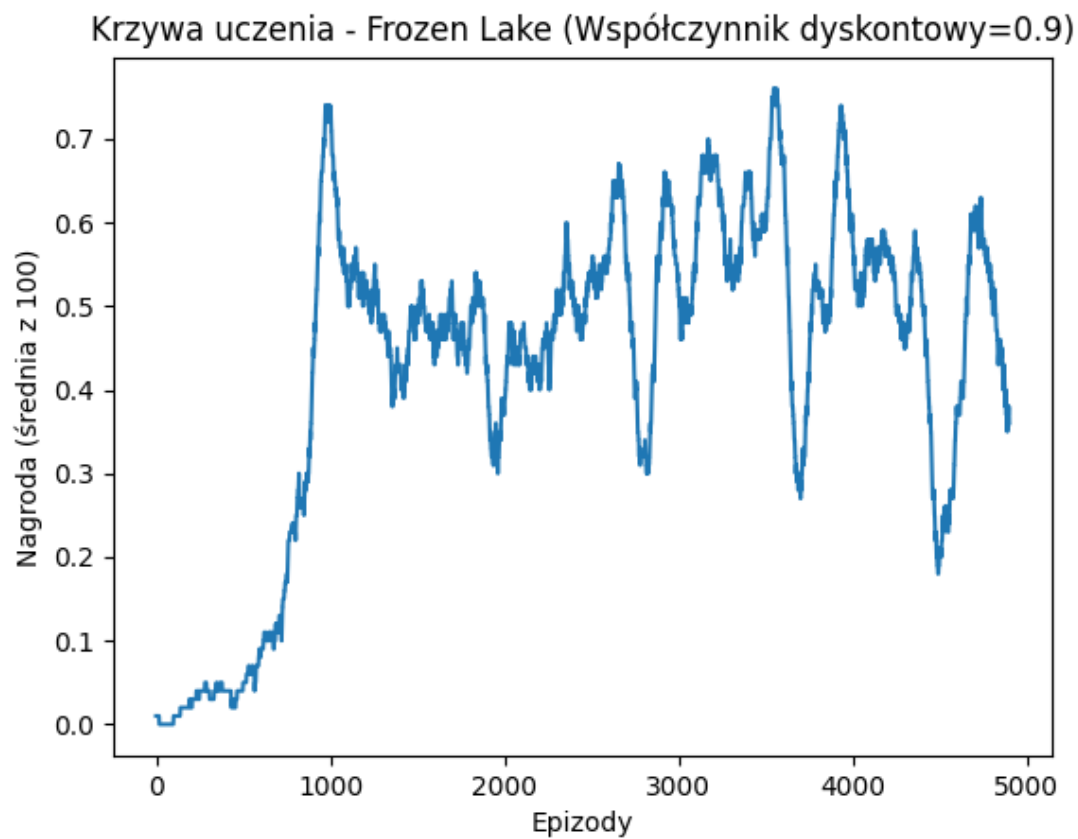
Rysunek 4: Krzywa uczenia dla eksperymentu 4

W tej wersji rozszerzonej, zmianie uległ współczynnik dyskontowy odpowiedzialny za patrzenie do przodu, z powyższego eksperymentu wynika iż jest on kluczowy dla sukcesu agenta, jego nieodpowiednia wartość może doprowadzić do nieefektywnego uczenia, jak widać na wykresie powyżej.

### 1.2.5 Eksperyment 5

Parametry:

- $\alpha = 0.1$ ,
- $\text{discount\_level} = 0.9$ ,
- $\epsilon = 1$ ,
- $\text{eps\_decay} = 0.001$ ,
- $\text{is\_slippery} = \text{True}$ ,
- $\text{episodes} = 5000$



Rysunek 5: Krzywa uczenia dla eksperymentu 5

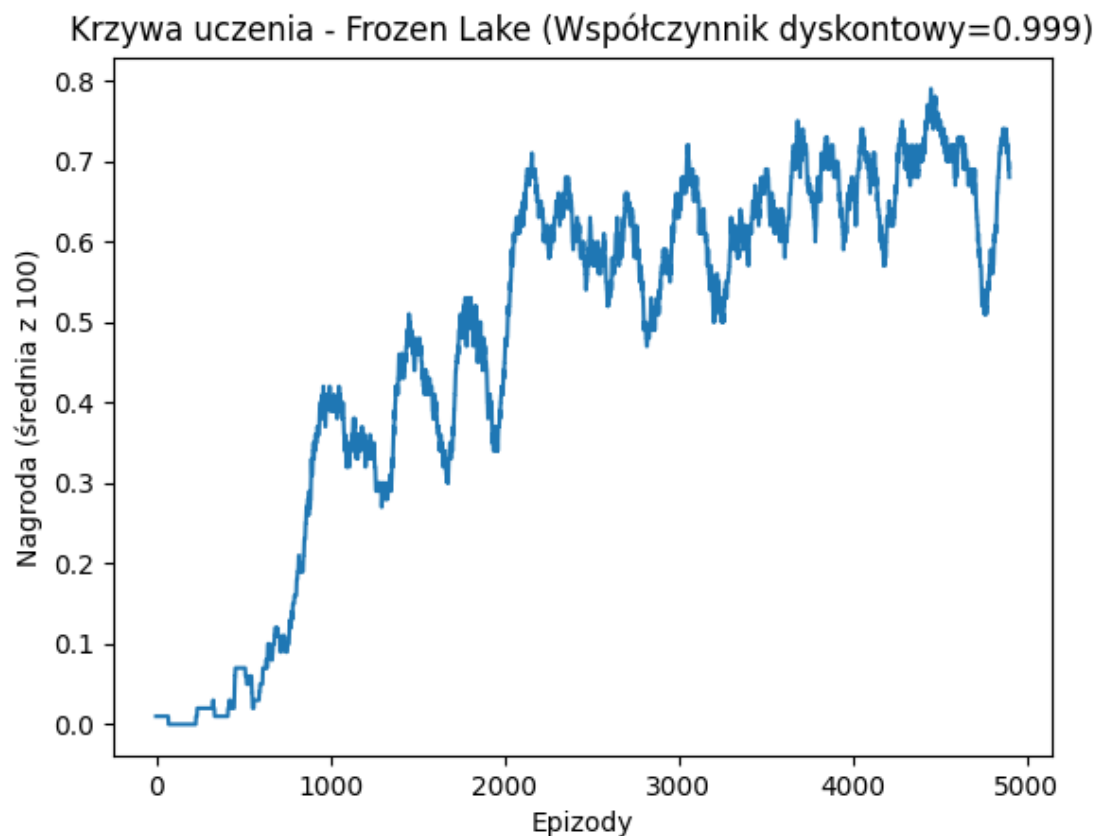
W tej wersji liczba epizodów uległa zwiększeniu, natomiast pozostałe parametry pozostają w optymalnej wersji. Widać, że po osiągnięciu szczytu możliwości około epizodu 1000, sprawność agenta zaczęła fluktuować.



### 1.2.6 Eksperyment 6

Parametry:

- $\alpha = 0.1$ ,
- $\text{discount\_level} = 0.999$ ,
- $\epsilon = 1$ ,
- $\text{eps\_decay} = 0.001$ ,
- $\text{is\_slippery} = \text{True}$ ,
- $\text{episodes} = 5000$



Rysunek 6: Krzywa uczenia dla eksperymentu 6

Jest to ten sam wariant co eksperyment wcześniej, lecz z małą zmianą w wielkości współczynnika dyskontowego, równego teraz 0.999. Powoduje to, że agent bardzo patrzy do przodu przy wyborze ruchu. Dla takiego zestawu parametrów agent potrafił uzyskać dużo lepsze wyniki, jednocześnie nie doświadczając tak dużych fluktuacji jak eksperyment wcześniej. Możliwe, że taka wartość współczynnika dyskontowego pozwoliła agentowi dostrzec ruchy, które zminimalizowały ryzyko przegranej

## 2 Gra z ciągłą przestrzenią obserwacji - CarRacing

### 2.1 Opis problemu

Jako problem o ciągłej przestrzeni obserwacji wybrano środowisko *CarRacing-v3*. Jest to zaawansowane zadanie sterowania, w którym agent uczy się prowadzenia samochodu wyścigowego na losowo generowanym torze. W przeciwieństwie do gry *Frozen Lake*, obserwacja w tym środowisku nie jest pojedynczą liczbą, lecz obrazem RGB o wymiarach 96x96 pikseli.

Agent kontroluje trzy parametry: skręt kierownicy (zakres -1 do 1), gaz (0 do 1) oraz hamulec (0 do 1). Cel gry to przejechanie całego toru w jak najkrótszym czasie. System nagród jest następujący: agent otrzymuje niewielką ujemną karę w każdym kroku czasowym za upływ czasu oraz dodatnie punkty za każdy nowo odwiedzony fragment trasy. Gra kończy się po przejechaniu wszystkich kafelków toru lub po upływie limitu czasu. Z tego powodu wielkość nagrody jest bardziej skomplikowana od zwykłego 0 i 1 jak w przypadku *Frozen Lake*, tutaj nagroda może wynieść dużo więcej.

### 2.2 Metodyka oraz eksperymenty

Ze względu na wysoką złożoność danych wejściowych, klasyczna tabela Q byłaby niemożliwa do zastosowania. Zastosowano algorytm **PPO**. Algorytm ten wykorzystuje sieci konwolucyjne (CNN), do interpretacji obrazu - ekstrakcji cech i na jej podstawie podejmuje decyzję w danej chwili, gdyż sieć zwraca dwie wartości dla każdej akcji -  $\mu$  oraz  $\sigma$ , czyli średnią i odchylenie standardowe, które zostaje wykorzystane do wybrania losowej wartości z utworzonego rozkładu gaussa. Dzieje się tak dla każdej z akcji.

Algorytm *PPO (Proximal Policy Optimization)* optymalizuje proces nauki poprzez zastosowanie mechanizmu *clippingu*. Mechanizm ten ogranicza gwałtowność aktualizacji polityki agenta, co zapobiega destabilizacji modelu w przypadku napotkania dziwnych stanów w środowisku. Proces uczenia odbywa się iteracyjnie: agent zbiera próbki doświadczenia, a następnie optymalizuje wagi sieci w taki sposób, aby zwiększyć prawdopodobieństwo akcji, które przyniosły nagrodę wyższą niż średnia przewidywana przez sieć Krytyka. Dzięki temu podejściu, agent stopniowo stabilizuje swój tor jazdy, ucząc się optymalnego punktu hamowania i przyspieszania na zakrętach.

W ramach eksperymentów skupiono się na analizie wpływu **współczynnika dyskontowego** ( $\gamma$ ) na proces nauki. Przetestowano zestaw trzech wartości:

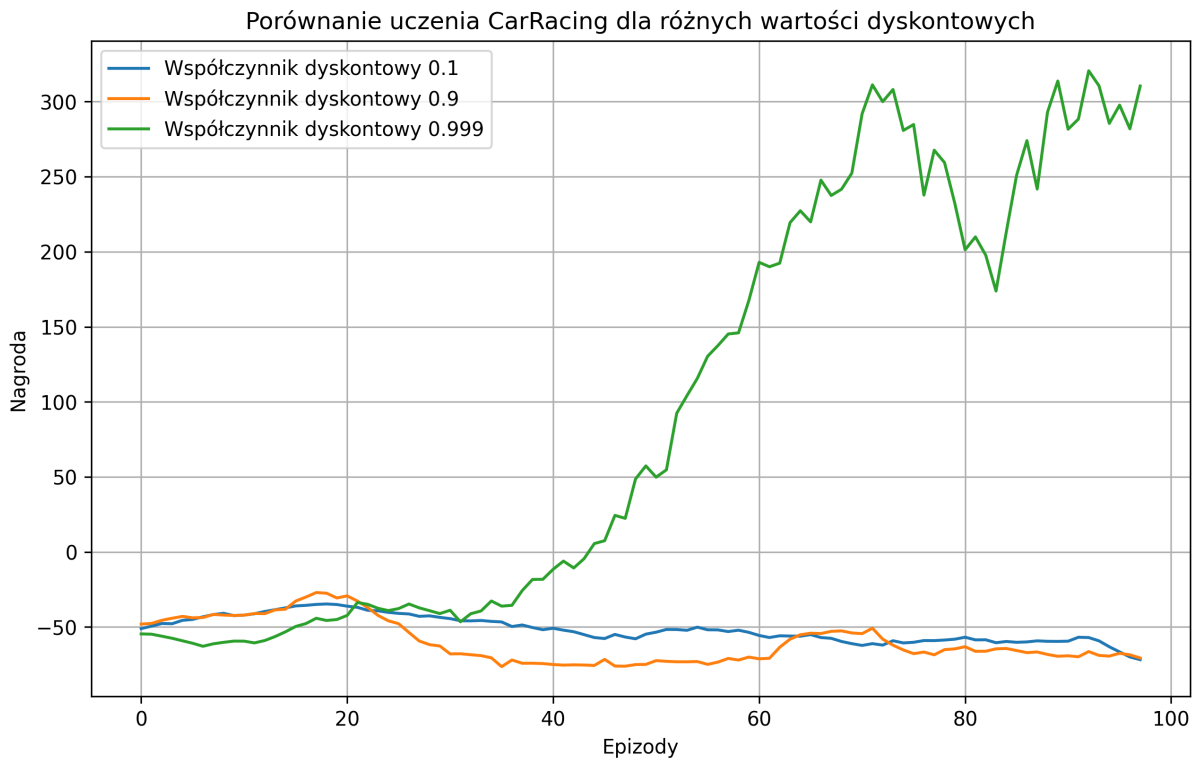
- $\gamma = 0.1$  – podejście "krótkowzroczne", gdzie agent skupia się niemal wyłącznie na natychmiastowych nagrodach,
- $\gamma = 0.9$  – podejście zbalansowane (bazowe), uwzględniające przyszłe korzyści z zachowaniem stabilności,
- $\gamma = 0.999$  – podejście "dalekowzroczne", w którym agent silnie bierze pod uwagę odległe w czasie nagrody, co ma kluczowe znaczenie przy planowaniu pokonywania długich zakrętów.

Wyniki zostały poddane wygładzeniu za pomocą średniej kroczącej, aby ułatwić rozczytanie trendu uczenia się agenta. Walidacja poprawności agenta była manualna - środowisko działało

w trybie renderowania widocznym dla człowieka.

Wyniki procesu nauczania:

- $\gamma = 0.1$ , średnia nagroda w procesie walidacyjnym = -85.92
- $\gamma = 0.9$ , średnia nagroda w procesie walidacyjnym = -76.92
- $\gamma = 0.999$ , średnia nagroda w procesie walidacyjnym = 257.95



Rysunek 7: Krzywe uczenia dla trzech wartości parametru dyskontowego

Wytlumaczenie - Dla dwóch pierwszych przypadków wyniki były niezadowalające, agent nie patrzył wystarczająco do przodu, nawet dla wartości 0.9, by móc podjąć realne działanie zbliżające go do sukcesu. Test manualny tylko to potwierdza - w pierwszym wypadku agent nie był w stanie nawet wystartować, w drugim przypadku agentowi udało się wystartować, lecz bardzo szybko wypadł z trasy i utknął w pętli nieskończonego "driftowania" w kółko. Dla ostatniej wartości agent zaczął od około 40 epizodu rozumieć środowisko na tyle dobrze, by poczynić postępy. Test manualny pokazał, że agent bardzo dobrze sobie radził na prostej trasie, natomiast miał lekkie problemy z zakrętami, które są trudniejsze i wymagają lepszego patrzenia do przodu, by nie wypaść z trasy.

## 3 Algorytmy uczenia ze wzmocnieniem

### 3.1 Wprowadzenie

W uczeniu ze wzmocnieniem występują 3 typy stosowanych algorytmów:

- **metoda programowania dynamicznego** (ang. *dynamic programming*, DP) - wartość stanu jest aktualizowana co krok w epizodzie dzięki znajomości prawdopodobieństwa przejść, metoda wykorzystywana w przypadku zadań epizodycznych (tj. posiadających ustalony stan końcowy);
- **metoda Monte Carlo** - dzięki próbkowaniu otrzymanych wartości w każdym kroku, obliczona na końcu epizodu wartość stanu jest podobna do tej wyliczonej za pomocą DP, metoda wykorzystywana w przypadku zadań epizodycznych;
- **metoda różnic tymczasowych** (ang. *temporal difference*, TD) - połączenie powyższych podejść, możliwe zastosowanie także w przypadku zadań ciągłych; metoda łączy aktualizację wartości stanu co krok z nieznanymi prawdopodobieństwami.

Aby dalsza analiza była możliwie pełna i kompleksowa zdecydowano się na implementację podejść **Value Iteration** (z kategorii metod programowania dynamicznego) oraz **Monte Carlo Control** (z kategorii metod Monte Carlo). Wykorzystywany wcześniej algorytm **Q-Learning** jest metodą należącą do grupy różnic tymczasowych, co pozwoliło na zestawienie ze sobą wszystkich 3 dominujących podejść. Do eksperymentów ponownie wykorzystano środowisko *Frozen Lake*.

### 3.2 Algorytm Q-Learning

Ten algorytm został szczegółowo opisany w sekcji 1.2.

Wyniki eksperymentów i komentarze o jego hiperparametrach są opisane w sekcjach 1.2.1 i kolejnych.

### 3.3 Algorytm Value Iteration

#### 3.3.1 Wstęp teoretyczny

Jak zostało powiedziane we wstępie, metoda ta jest przykładem metod programowania dynamicznego. Algorytm zakłada znajomość modelu środowiska (przejść i nagród), co ma miejsce w przypadku gry *Frozen Lake*. Intuicją stojącą za tym podejściem jest obliczenie optymalnej strategii z góry zamiast nauczania się jej przez kolejne próby.

Konieczna jest znajomość:

- $P(s'|s, a)$  - prawdopodobieństwo przejść,
- $R(s, a, s')$  - nagród

Rozważane jest tzw. **Markov Decision Process (MDP)**, czyli piątka

$$(S, A, P, R, \gamma),$$

gdzie:

- $S$  - zbiór stanów,
- $A$  - zbiór akcji,
- $P(s'|s, a)$  - prawdopodobieństwo przejścia,
- $R(s, a, s')$  - nagroda,
- $\gamma \in [0, 1)$  - współczynnik dyskontowania

Definiuje się także **operator optymalności Bellmana**:

$$(TV)(s) = \max_{a \in A} \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V(s')]$$

Algorytm generuje ciąg funkcji

$$V_{k+1}(s) = TV_k,$$

czyli

$$V_{k+1}(s) = \max_{a \in A} \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_k(s')].$$

Pozwala to na zdefiniowanie **funkcji optymalnej**

$$V^*(s) = \max_{a \in A} \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')],$$

która bezpośrednio przekłada się na **politykę optymalną**

$$\pi^*(s) \in \arg \max_{a \in A} \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')].$$

**Value Iteration** może więc być interpretowana jako iteracyjne stosowanie operatora Bellmana, które zbiega do jedynego rozwiązania równania optymalności. Algorytm działa iteracyjnie aż uzyska zbieżność; następnie polityka to wybór akcji maksymalizującej  $V(s)$ . Podejście to może być zastosowane w przypadku środowiska *Frozen Lake* ze względu na pełną znajomość modelu; możliwe jest także uwzględnienie parametru śliskości przez bazowanie na prawdopodobieństwach  $P(s'|s, a)$ .

W ogólności ta metoda ma jednak wady i zalety:

- **zalety:**
  - daje optymalne rozwiązanie,
  - szybko działa dla małych środowisk;
- **wady:**
  - wymaga znajomości modelu, co często jest nieosiągalne,
  - słabo się skaluje.

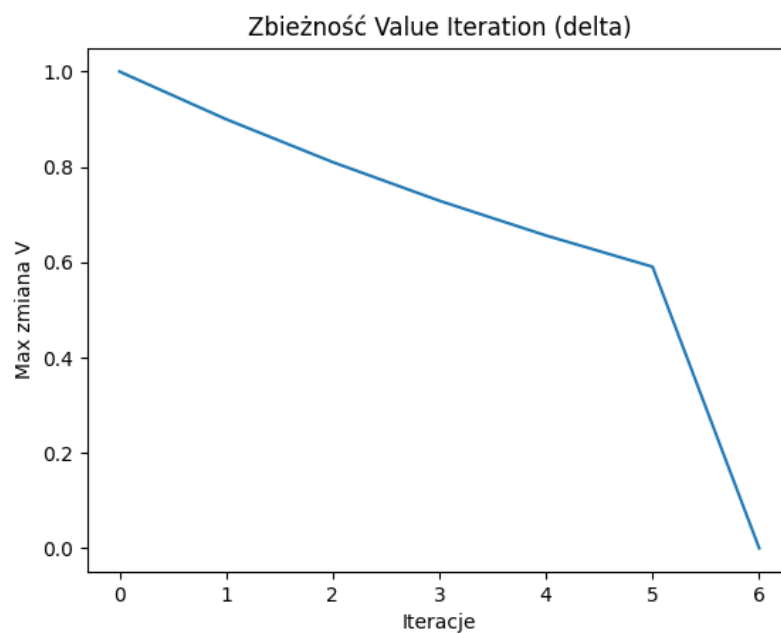
### 3.3.2 Analiza działania

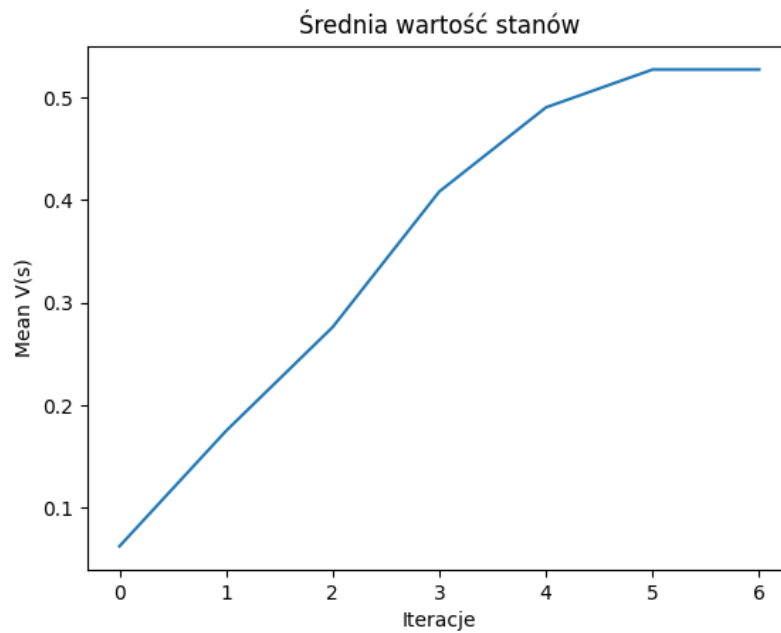
W 1 eksperymencie algorytm został uruchomiony na grze bez poślizgu (`is_slippery=False`) ze współczynnikiem dyskontowym równym 0.9. Wykres nagrody dla wyliczonej polityki działania był następujący.



Jak widać algorytm bezbłędnie poradził sobie w deterministycznym środowisku w każdym kroku dochodząc do celu. Metoda Value Iteration najpierw policzyła optymalną politykę, która omija dziury i prowadzi do celu, a następnie sukcesywnie ją stosowała.

Dalsze analizy w formie wykresów zbieżności algorytmu oraz średniej wartości stanów również potwierdzają jego poprawne działanie.





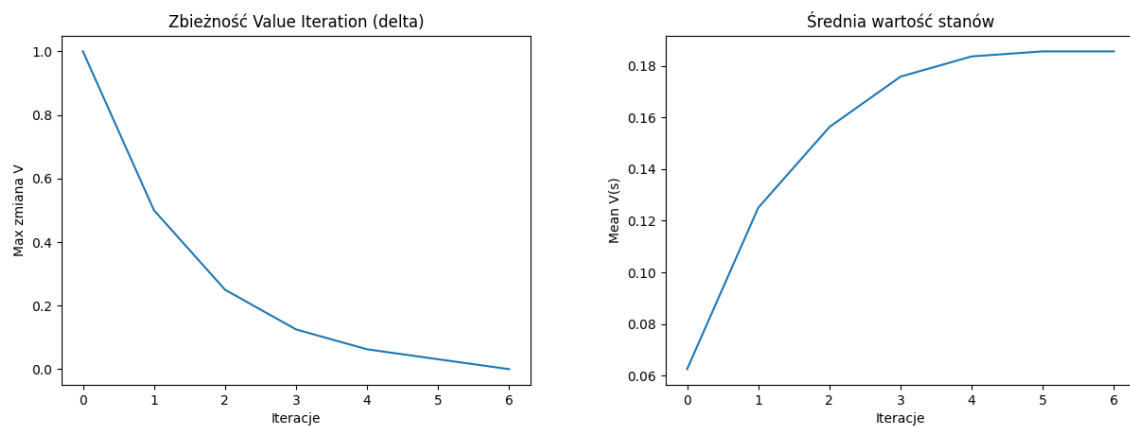
Widać, że błąd między kolejnymi iteracjami maleje do zera, co dowodzi zbieżności. Średnia wartość stanów zbiega do wartości równej około 0.5, co również jest zgodne z oczekiwaniami. Mierzona tutaj wartość to bowiem  $V(s)$ , które jest oczekiwanym zdyskontowany, zwrotem, a nie nagroda końcową

$$V(s) \approx \gamma^d,$$

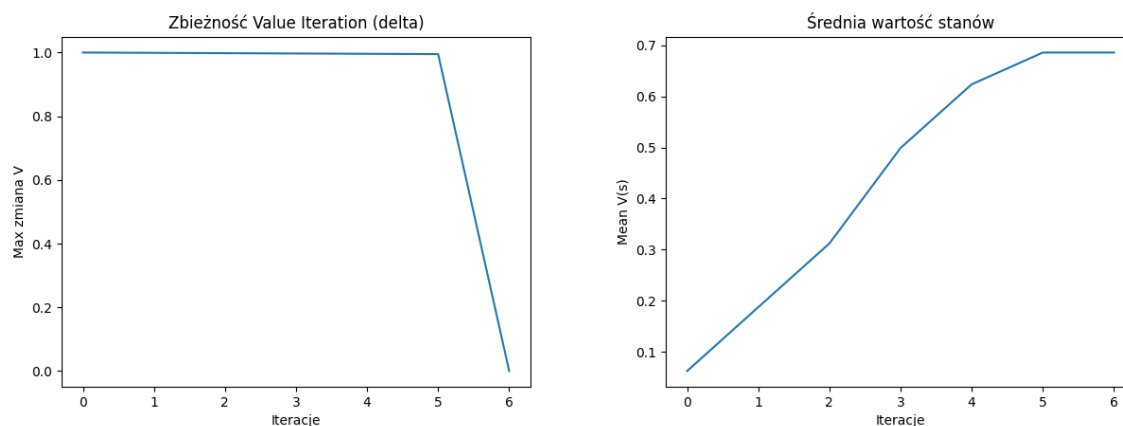
gdzie  $d$  to liczba kroków. Na siatce *Frozen Lake*  $4 \times 4$  średnia liczba kroków potrzebnych do dojścia do celu to 6, więc

$$V(s) \approx 0.9^6 \approx 0.53.$$

W dalszych eksperymentach, powtarzając rozważania z 1 rozdziału, sprawdzono zachowanie algorytmu dla współczynnika dyskontowego równego 0.5 i 0.999.



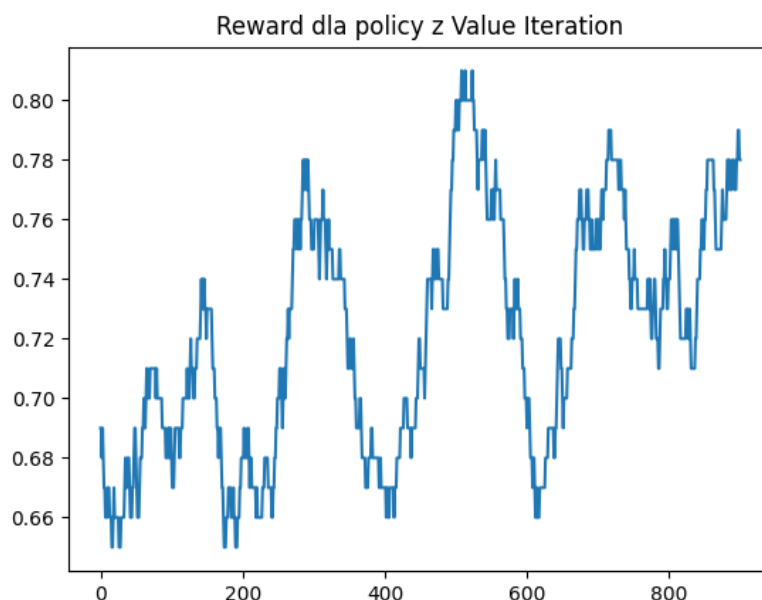
Rysunek 8: Wykresy dla współczynnika dyskontowego równego 0.5



Rysunek 9: Wykresy dla współczynnika dyskontowego równego 0.999

Jak widać w obu przypadkach błąd między kolejnymi iteracjami zmalał do zera, dla wartości 0.999 spadek był bardziej gwałtowny. Średnia wartość stanów w przypadku współczynnika dyskontowego została ustalona na poziomie 0.18, co pokazuje że algorytm potrzebował większej liczby kroków niż w przypadku wartości 0.9. Dla wartości 0.999 średnia wartość była jeszcze wyższa, na poziomie bliskim 0.7. Potwierdza to intuicyjne znaczenie parametru dyskontowego. Niezależnie od jego wartości (0.5, 0.9 czy 0.999) algorytm zawsze ustalał w pełni skuteczną politykę.

Wprowadzenie do gry *Frozen Lake* niedeterminizmu w postaci poślizgu (`is_slippery=True`) wprowadza dodatkową trudność dla algorytmu.



Widać wyraźne wahania w wartościach uzyskiwanych przez algorytm, jednak niemal wszystkie utrzymują się na poziomie ponad 0.68, co daje mu lekką przewagę nad podejściem Q-Learning. Natura niedeterministycznej gry z poślizgiem uniemożliwia osiągnięcie idealnych wyników, więc uzyskane rezultaty można przyjąć za zadowalające.



## 3.4 Algorytm Monte Carlo Control

### 3.4.1 Wstęp teoretyczny

Metoda ta, jak sama nazwa wskazuje, należy do grupy metod Monte Carlo. Nauka polega na analizie całych epizodów i przypisaniu zasługi wcześniejszym decyzjom, które wpłynęły na końcowy wynik; nie zakłada się znajomości  $P$  i  $R$  (prawdopodobieństw i nagród). Mechanizm jest następujący:

1. Epizod jest uruchamiany aż do końca.
2. Zbiera się sekwencję stan - akcja - zwrot, tj.  $(s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_N)$ .
3. Liczy się całościowy zwrot z wykorzystaniem współczynnika dyskontowego:  
$$G_t = r_{t+1} + \gamma r_{t+1} + \dots$$
4. Aktualizacja  $Q(s, a)$  następuje w oparciu o średnią z zaobserwowanych  $G_t$ .

Po tym wszystkim następuje **estymacja funkcji wartości akcji** dla danej polityki  $\pi$

$$Q^\pi(s, a) = E_\pi[G_t | s_t = s, a_t = a].$$

Następnie polityka jest wyciągana jako

$$\pi(s) \in \arg \max_{a \in A} Q(s, a)$$

W przypadku środowiska *Frozen Lake* epizody są dość krótkie, ale nagroda występuje jedynie na ich końcach, więc sygnał zwrotny jest dość rzadki, co może powodować wolniejszy proces nauki niż np. w przypadku Q-Learningu. Metoda ta ma też naturalnie swoje ogólne cechy charakterystyczne:

- **zalety:**
  - prostota konceptualna,
  - brak konieczności znajomości modelu;
- **wady:**
  - duża wariancja,
  - przez konieczność czekania aż do końca epizodu możliwy wolniejszy proces nauki.

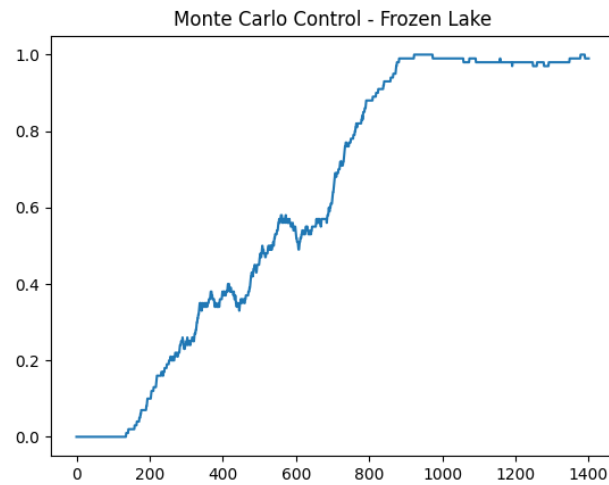
### 3.4.2 Analiza działania

Analogiczny zestaw hiperparametrów w porównaniu z algorytmem Q-Learning umożliwił przeprowadzenie wszystkich 6 eksperymentów i porównanie działania obu algorytmów.

#### 1. eksperyment 1:

- **hiperparametry:**  
alpha=0.5, discount\_level=0.9, epsilon=1, eps\_decay=0.001, is\_slippery=False, episodes=1500

- **wyniki:**



Rysunek 10: Monte Carlo Control - eksperyment 1

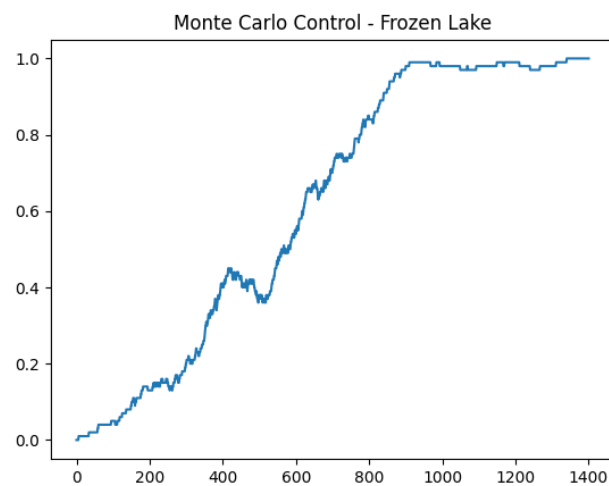
- **porównanie z QL:** wyniki bardzo zbliżone

## 2. eksperyment 2:

- **hiperparametry:**

$\alpha=0.1$ ,  $\text{discount\_level}=0.9$ ,  $\text{epsilon}=1$ ,  $\text{eps\_decay}=0.001$ ,  $\text{is\_slippery}=\text{False}$ ,  $\text{episodes}=1500$

- **wyniki:**



Rysunek 11: Monte Carlo Control - eksperyment 2

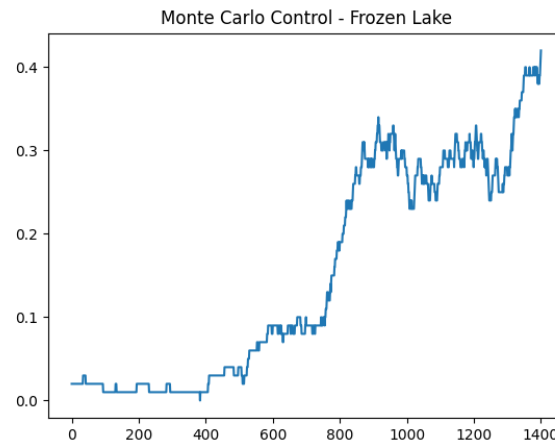
- **porównanie z QL:** wyniki bardzo zbliżone

### 3. eksperyment 3:

- **hiperparametry:**

$\alpha=0.1$ ,  $\text{discount\_level}=0.9$ ,  $\text{epsilon}=1$ ,  $\text{eps\_decay}=0.001$ ,  $\text{is\_slippery}=\text{True}$ ,  $\text{episodes}=1500$

- **wyniki:**



Rysunek 12: Monte Carlo Control - eksperyment 3

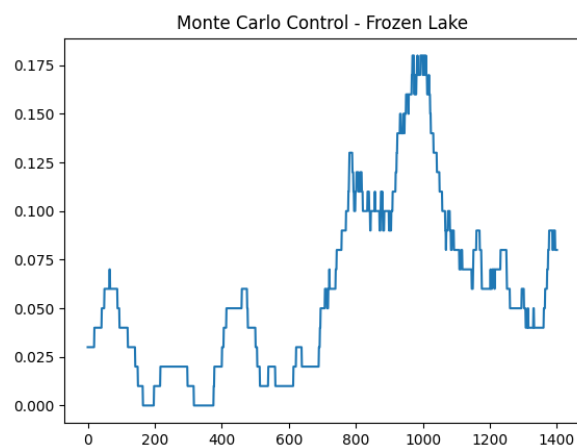
- **porównanie z QL:** wyniki wyraźnie gorsze

### 4. eksperyment 4:

- **hiperparametry:**

$\alpha=0.1$ ,  $\text{discount\_level}=0.5$ ,  $\text{epsilon}=1$ ,  $\text{eps\_decay}=0.001$ ,  $\text{is\_slippery}=\text{True}$ ,  $\text{episodes}=1500$

- **wyniki:**



Rysunek 13: Monte Carlo Control - eksperyment 4

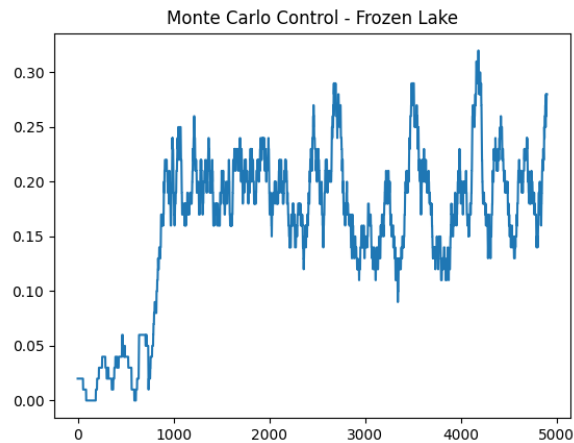
- **porównanie z QL:** wyniki gorsze

## 5. eksperyment 5:

- **hiperparametry:**

$\alpha=0.1$ ,  $\text{discount\_level}=0.9$ ,  $\text{epsilon}=1$ ,  $\text{eps\_decay}=0.001$ ,  $\text{is\_slippery}=\text{True}$ ,  $\text{episodes}=5000$

- **wyniki:**



Rysunek 14: Monte Carlo Control - eksperyment 5

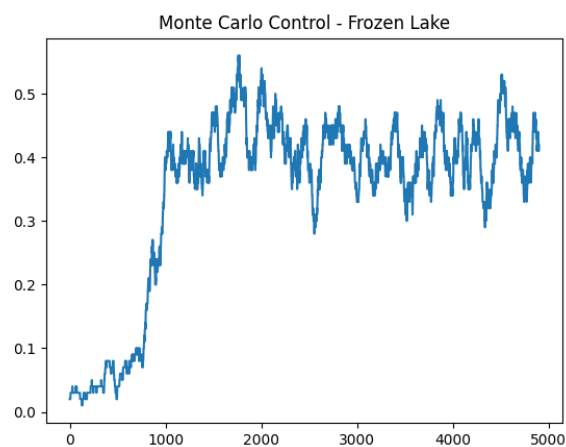
- **porównanie z QL:** wyniki znacząco gorsze

## 6. eksperyment 6:

- **hiperparametry:**

$\alpha=0.1$ ,  $\text{discount\_level}=0.999$ ,  $\text{epsilon}=1$ ,  $\text{eps\_decay}=0.001$ ,  $\text{is\_slippery}=\text{True}$ ,  $\text{episodes}=5000$

- **wyniki:**



Rysunek 15: Monte Carlo Control - eksperyment 6

- **porównanie z QL:** wyniki gorsze

Ogólnie algorytm Monte Carlo Control wypadł słabo w porównaniu z Q-Learningiem. Osiągał zbliżone wartości dla deterministycznej wersji gry i kompletnie poległ w przypadku niedeterministycznym. Osiągnął też słabe rezultaty w zestawieniu z algorytmem Value Iteration.

### 3.5 Strojenie hiperparametrów

W ostatniej sekcji analizy algorytmów zostały one poddane strojeniu hiperparametrów. Zastosowano optymalizację ze względu na całkowitą zdyskontowaną nagrodę w 1000 pierwszych epizodach lub krokach. Formalnie jest to maksymalizacja wyrażenia

$$J(\theta) = \sum_{t=0}^{T=1000} \gamma^t r_t$$

Zastosowano metodę GridSearch.

#### 3.5.1 Q-Learning

Spośród analizowanych parametrów, tj.:

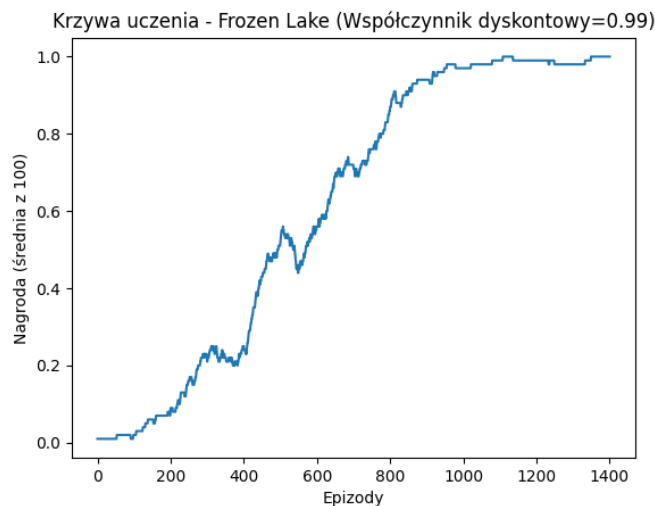
- `alphas` = [0.1, 0.3, 0.5],
- `gammas` = [0.8, 0.9, 0.99],
- `epsilons` = [1.0, 0.5],
- `decays` = [0.001, 0.0005]

przeszukiwanie GridSearch wskazało jako optymalny zestaw:

- bez poślizgu: `alpha=0.5`, `gamma=0.99`, `epsilon=1`, `decay=0.001`
- z poślizgiem: `alpha=0.5`, `gamma=0.99`, `epsilon=0.5`, `decay=0.001`

Dla takich zestawów parametrów wykonano analizy zbieżności:

#### 1. bez poślizgu:



## 2. z poślizgiem:



### 3.5.2 Value Iteration

Spośród analizowanych parametrów, tj.:

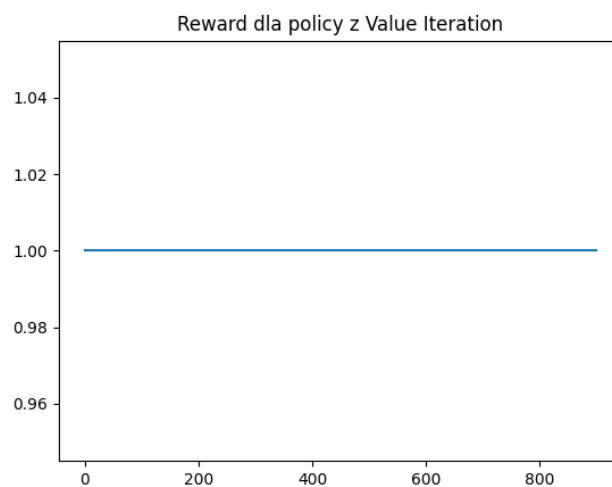
- `gammas = [0.5, 0.7, 0.9, 0.99]`

Przeszukiwanie GridSearch wskazało, że

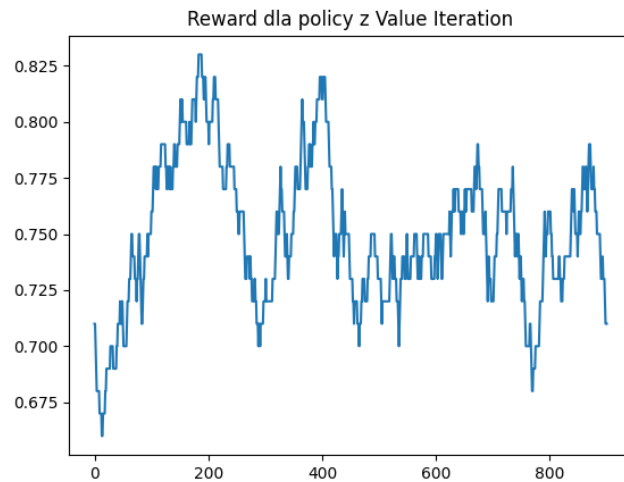
- w przypadku braku poślizgu każdy ze współczynników doprowadza algorytm do 100% skuteczności,
- w przypadku z poślizgiem najlepsza wartość gamma to 0.99.

Dla takich wartości parametru wykonano analizy zbieżności:

#### 1. bez poślizgu:



## 2. z poślizgiem:



### 3.5.3 Monte Carlo Control

Spośród analizowanych parametrów, tj.:

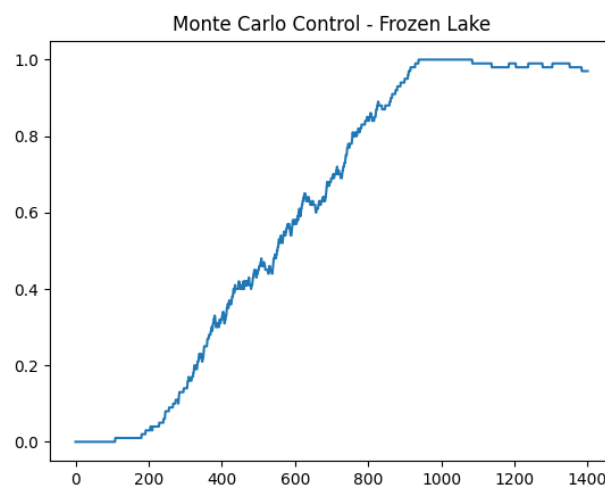
- `gammas = [0.8, 0.9, 0.99]`,
- `epsilons = [1.0, 0.5]`,
- `decays = [0.001, 0.0005]`

przeszukiwanie GridSearch wskazało jako optymalny zestaw:

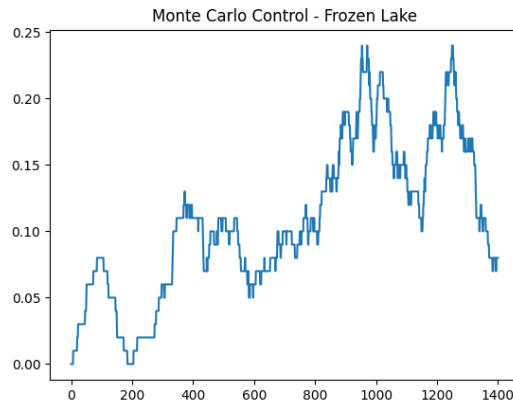
- bez poślizgu: `gamma=0.99`, `epsilon=1`, `decay=0.001`
- z poślizgiem: `gamma=0.99`, `epsilon=0.5`, `decay=0.0005`

Dla takich zestawów hiperparametrów wykonano analizy zbieżności:

#### 1. bez poślizgu:



## 2. z poślizgiem:



### 3.6 Podsumowanie

Celem przeprowadzonych eksperymentów było porównanie wybranych algorytmów uczenia ze wzmocnieniem na przykładzie środowiska *Frozen Lake*. Analizie poddano trzy podejścia: Q-learning, Value Iteration oraz Monte Carlo Control, uwzględniając zarówno ich własności teoretyczne, jak i ich faktyczne zachowanie.

**Value Iteration** należy do metod programowania dynamicznego i zakłada znajomość modelu środowiska. Algorytm iteracyjnie rozwiązuje równanie optymalności, co gwarantuje zbieżność do optymalnej funkcji wartości, a tym samym do optymalnej polityki. Jego główną zaletą jest deterministyczność oraz szybka i stabilna zbieżność w małych, dyskretnych środowiskach. Wadą pozostaje konieczność znajomości modelu, co ogranicza jego zastosowanie w praktyce.

**Q-learning** jest metodą bezmodelową. Uczy się funkcji wartości akcji na podstawie próbek interakcji ze środowiskiem, dążąc do aproksymacji optymalnej polityki niezależnie od aktualnie wykonywanej strategii eksploracji. Algorytm ten jest bardziej uniwersalny niż Value Iteration, jednak jego skuteczność zależy od odpowiedniego doboru hiperparametrów (współczynnik uczenia, eksploracja, współczynnik dyskontowy), a proces uczenia może być niestabilny i wolniejszy.

**Monte Carlo Control** również jest metodą bezmodelową, jednak w przeciwieństwie do Q-learningu opiera się na pełnych epizodach. Aktualizacja wartości następuje na podstawie całkowitego zwrotu, co eliminuje bias, ale zwiększa wariancję estymacji. W efekcie algorytm ten charakteryzuje się wolniejszą zbieżnością oraz większą wrażliwością na rzadkie nagrody.

Przeprowadzone eksperymenty wykazały wyraźne różnice w skuteczności analizowanych metod. Value Iteration osiągnął najlepsze wyniki, znajdując optymalną politykę w sposób szybki i stabilny. W przypadku środowiska deterministycznego prowadziło to do uzyskania maksymalnej nagrody w każdym epizodzie. Q-learning uzyskał nieco słabsze rezultaty, wymagając większej liczby epizodów oraz odpowiedniego strojenia hiperparametrów. Mimo to był w stanie zbliżyć się do rozwiązania optymalnego. Monte Carlo Control wypadł najslabiej, co wynika z wysokiej wariancji estymacji oraz konieczności uczenia się na podstawie rzadkich sygnałów nagrody.

Uzyskane wyniki są zgodne z oczekiwaniami teoretycznymi. Metody wykorzystujące pełną wiedzę o modelu środowiska (Value Iteration) osiągają najlepsze rezultaty w prostych, dyskretnych problemach. Algorytmy bezmodelowe, takie jak Q-learning i Monte Carlo, oferują większą uniwersalność, jednak kosztem efektywności i stabilności uczenia.